

Batch vs Stream Processing

Phase 3 · Advanced · Estimated time: 2 hours

1. Concept From First Principles

Chapter 6's ETL pipeline was described somewhat vaguely as "a scheduled job" moving data into a warehouse. This chapter makes that precise by covering the two fundamentally different ways large volumes of data actually get processed: in scheduled, bounded **batches**, or continuously, as an **unbounded stream**. Which one you choose has major consequences for latency, complexity, and cost, and real systems almost always need both, for different purposes.

Batch processing treats data as a bounded, finite dataset: you accumulate records over some window (an hour, a day) or until you hit some volume, then process the entire batch together, typically on a fixed schedule (a nightly cron job, an hourly pipeline run). **MapReduce**, the programming model that popularized large-scale batch processing (originally at Google, later open-sourced as part of Hadoop), captures the essential shape: a **Map** phase processes and transforms input records independently and in parallel across many machines (stateless, one record at a time), and a **Reduce** phase aggregates the mapped results together (summing revenue per region, counting occurrences per key). This map-then-reduce shape lets a batch job scale out across a large cluster with relatively little custom coordination logic, since the Map phase is naturally parallelizable.

Stream processing treats data as unbounded and continuous: instead of waiting to accumulate a batch, each event (or a small **micro-batch** of events) is processed individually, as it arrives, producing output in near-real time. Frameworks like **Kafka Streams** and **Apache Flink** build on top of the message queues from Phase 2 Chapter 8, consuming a continuous stream of events and applying transformations, aggregations, and **windowing** (grouping events into time-bounded chunks, like "count events in each rolling 5-minute window") without ever needing a full, bounded dataset to work from.

Backend engineer analogy

A nightly Laravel scheduled command (`php artisan schedule:run` triggering a job that processes yesterday's full order log) is classic batch processing — bounded, scheduled, working on a complete dataset. A queue worker (`php artisan queue:work`) consuming from a Redis/SQS queue and processing each job the moment it arrives is much closer to stream processing in spirit — continuous, per-event, no waiting for a full batch to accumulate. Most systems you've already built use a mix of both patterns without necessarily naming them this way.

Choosing between them: latency vs. complexity

The core trade-off is latency versus operational complexity. Batch processing is simpler to reason about, easier to debug (you can re-run a batch job against the same bounded input and get the same output, which stream processing's continuous, ordering-sensitive nature makes much harder), and well-suited to workloads where some delay (minutes to a day) is entirely acceptable — nightly reports, ML model training on historical data, warehouse ETL (Chapter 6). Stream processing is considerably more complex to build and operate correctly — it must handle out-of-order events (Chapter 1's

clock-skew problem again), late-arriving data, and maintain running state (like a rolling count) reliably across failures — but it's the only option when the business genuinely needs near-real-time results: fraud detection that must flag a suspicious transaction within seconds, not the next morning; live dashboards; real-time recommendation updates.

A useful mental model, sometimes called the **Lambda architecture** in its classic form, runs both paths side by side deliberately: a stream-processing path gives fast, approximate, real-time results, while a batch-processing path periodically recomputes the same metrics exactly and correctly over the full historical dataset, reconciling any drift the fast path accumulated. Newer approaches (sometimes called Kappa architecture) instead try to handle everything through a single stream-processing path with sufficiently robust replay capability, avoiding the operational cost of maintaining two separate pipelines — the trade-off between these two architectural philosophies is itself a live, debated topic worth being aware of rather than memorizing as settled.

It's worth being explicit, since interviewers probe this directly: choosing stream processing isn't free just because "real-time sounds better." The added complexity of correctly handling out-of-order events, windowing semantics, and stateful fault tolerance is a genuine engineering cost that should be justified by a real business requirement for low latency, not adopted by default. A system genuinely fine with next-morning reports should stick with the simpler, cheaper, easier-to-debug batch approach.

2. Real-World Example / Case Study

Razorpay's and PhonePe's fraud-detection systems need stream processing almost by definition — a fraudulent transaction pattern flagged an hour later, in a nightly batch report, has already caused damage; the value of fraud detection is directly tied to how quickly a suspicious pattern (an unusual spending spike, a rapid sequence of failed payment attempts) can be caught and acted on, which only stream processing can deliver at the needed latency.

Flipkart's and Swiggy's nightly business-intelligence and inventory-reconciliation jobs — computing exact revenue totals, restocking recommendations, and settlement figures across the full previous day's activity — are naturally suited to batch processing, since a report ready the next morning is entirely acceptable, and batch processing's simpler, more auditable, exactly-reproducible nature is genuinely preferable for financial and inventory accuracy.

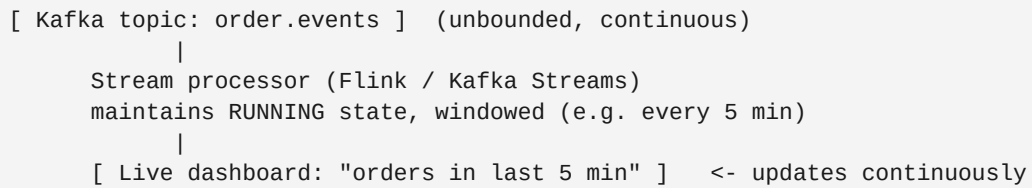
Careem's and Talabat's live-tracking and surge-pricing systems, needing to react to current demand and driver-availability patterns within seconds, are a stream-processing use case — a surge-pricing algorithm computed from yesterday's demand pattern would be useless for responding to a demand spike happening right now during a sudden rainstorm or major event.

3. Diagram

Batch processing (MapReduce shape):



Stream processing (continuous, windowed):



4. Trade-offs Table

Aspect	Batch processing	Stream processing
Latency	Minutes to a day	Sub-second to a few seconds
Complexity	Lower — simpler to reason about and debug	Higher — out-of-order events, windowing, state management
Reproducibility	Exact, easily re-run on the same bounded input	Harder — continuous, ordering-sensitive
Typical tools	Hadoop/MapReduce, Spark batch jobs	Kafka Streams, Apache Flink, Spark Streaming
Good fit	Reports, ML training, warehouse ETL (Ch 6)	Fraud detection, live dashboards, surge pricing

5. Common Interview Questions

Q1: "When would you choose stream processing over batch, and what does it cost you?"

Model approach: Tie the choice to a genuine, stated latency requirement (seconds, not hours), and name the real cost explicitly — handling out-of-order events, windowing complexity, and stateful fault tolerance — making clear that stream processing shouldn't be a default choice without that justification.

Q2: "Explain the Map and Reduce phases of MapReduce with a concrete example."

Model approach: Use a word-count or revenue-aggregation example — Map processes each record independently and in parallel (extract region + amount from each order), Reduce aggregates mapped results by key (sum amount per region) — and explain why the Map phase's independence is what makes large-scale parallelism straightforward.

Q3: "What does 'windowing' mean in stream processing, and why is it needed?"

Model approach: Explain that an unbounded stream has no natural end point to aggregate over, so windowing artificially bounds it into time-based chunks (e.g., a rolling 5-minute window) to make aggregate computations like counts or sums meaningful and computable.

Q4: "What is the Lambda architecture, and what problem does it solve?"

Model approach: Describe running a fast, approximate stream-processing path alongside a slower, exact batch-processing path that periodically reconciles the same metrics, solving the tension between real-time responsiveness and exact correctness, while acknowledging the added operational cost of maintaining two pipelines (and mentioning Kappa architecture as a stream-only alternative).

6. Practice Exercises

- 1 Classify five real features (a fraud alert, a monthly invoice, a live 'orders in the last hour' dashboard, a nightly inventory sync, a real-time chat unread counter) as batch or stream, with a one-line justification each.
- 2 Sketch a MapReduce job that computes total revenue per restaurant from a day's order log, describing what the Map and Reduce phases each do.
- 3 Explain, using a concrete example, why an event arriving 'late' (its timestamp is earlier than events already processed) is a genuine problem for a windowed stream aggregation, and how you might handle it.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"DDIA Chapter 11 - Stream Processing The Big Ideas"	Grounded in Designing Data-Intensive Applications' treatment of streaming.
English	"Stream Processing 101 - An Introduction"	Accessible introduction to core stream-processing concepts.
Hindi	"System Design Complete Course 2026" (Hindi + English, 10+ hrs)	Covers batch and stream processing as part of a full bilingual course.
Hindi	"Complete System Design Roadmap 2026 HLD & LLD" — CodeHelp	Places batch/stream processing in context of the broader Hindi HLD roadmap.

8. Key Takeaways

- Batch processing works on bounded, scheduled datasets; stream processing works continuously on unbounded data.
- MapReduce's Map (parallel, stateless transform) and Reduce (aggregate) phases are the core batch-processing shape.
- Stream processing frameworks (Kafka Streams, Flink) build on message queues (Phase 2, Ch 8) for near-real-time results.
- Windowing artificially bounds an unbounded stream into time-based chunks to make aggregation meaningful.
- Stream processing trades simplicity for latency — it's genuinely harder to build and debug correctly.

- Lambda architecture runs both paths side by side; Kappa architecture tries to unify on stream processing alone.
- Choose stream processing only when a real, stated latency requirement justifies its added operational complexity.